SMICES | 2026–2027

AI for Cybersecurity

C3 | Deep Neural Networks

Ana Nicolae, Vlad Vasilescu
28 October 2026

National University of Science and Technology POLITEHNICA Bucharest
Faculty of Electronics, Telecommunications and Information Technology

Overview

1 Intro

2 Dense Neural networks

3 Differntial programming

4 Gradient descent algorithms

5 Initialization mechanisms

Radial Basis Function Nets

Consider pairs $(x, y) \in \mathbb{R}^d$.

- 1-layer architecture, which learns means and variances $\forall i \in [0 \dots N-1] \quad (\mu_i, \sigma_i)$ as "weights".

$$\phi(x) = \begin{pmatrix} 1 \\ \exp \frac{-||x-\mu_0||^2}{2\sigma_0^2} \\ \vdots \\ \exp \frac{-||x-\mu_{N-1}||^2}{2\sigma_{N-1}^2} \end{pmatrix} \tag{1}$$

- we model $y = g(w^\top \phi(x))$, where g is called the transfer function
- can be used to solve different problems like: *regression*, *binary classification* and *multi-classification*.

Learning mechanism

- Place the centers and variances randomly and uniformly, by vector quantization (K-means++, GNG, etc.)

2-steps learning

1. fit the centers (μ_i) and variances (σ_i).
2. fit the weights w.

3-steps learning

1. fit the centers (μ_i) and variances (σ_i).
2. fit the weights w.
3. fit everything ($\nabla_\mu \mathcal{L}, \nabla_\sigma \mathcal{L}, \nabla_w \mathcal{L}$), where $\mathcal{L}$ is the cost function.

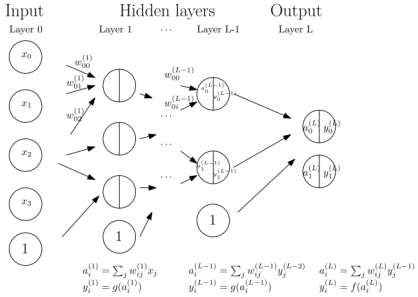
Universal approximator theorem

RBFN are **universal approximators**. Denote $\mathcal{S}$ the family of functions based on RBF in $\mathbb{R}^d$:

$$\mathcal{S} = \{\zeta : \mathbb{R}^d \to \mathbb{R}, \zeta(x) = \sum_i \Theta\left(\frac{x - \mu_i}{\sigma_i}\right), w \in \mathbb{R}^N\},$$

with $\Theta : \mathbb{R}^d \to \mathbb{R}$ continuous (almost everywhere) and $\int_{\mathbb{R}^d} \Omega(x)dx \neq 0$.
Then $\mathcal{S}$ is dense in $\ell_p(\mathbb{R})$ for every $p \in [1, \infty)$.

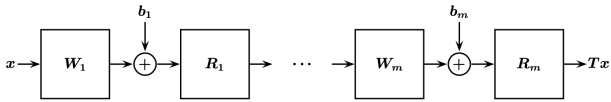
Feedforward Neural Network Architecture



Notations

- **Width** : number of neurons per layer
- **Depth** : number of weight layer
- **Parameters** : weight matrices and bias vector associated with every layer
- **Activation** : a hidden (non-linear) transfer function, denoted with f for layers $l \in \{1 \dots L-1\}$ and g for the last layer L.

Feed-forwars neural networks



An m layers FCN network (T) can be viewed as a composition of elementary functions modelling layers:

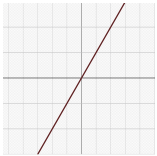
$$T = T_m \circ \cdots \circ T_1.$$

$(\forall i \in \{1, \ldots, m\})$ $\qquad T_i(x_i) = R_i(W_i^\top x_i + b_i)$

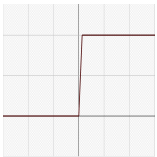
$x_i \in \mathbb{R}^{N_{i-1}}$ – input of the layer
$b_i \in \mathbb{R}^{N_i}$ – bias parameter,
$W_i \in \mathbb{R}^{N_i \times N_{i-1}}$ – weight matrix,
$R_i \colon \mathbb{R}^{N_i} \to \mathbb{R}^{N_i}$ – activation operator.

Classic Activation Operators

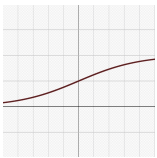
- Identity: $\rho : \mathbb{R} \to \mathbb{R} \quad \rho(x) = x$



- Binary step: $\rho : \mathbb{R} \to \{0, 1\} \quad \rho(x) = \begin{cases} 0, & \text{if} \quad x < 0 \\ 1, & \text{if} \quad x \geq 0 \end{cases}$

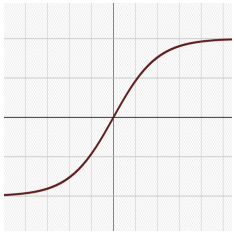


- Sigmoid: $\rho : \mathbb{R} \to (0, 1) \quad \rho(x) = \frac{1}{1+e^{-x}}$

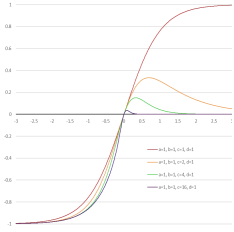


Classic Activation Operators

- Hyperbolic tangent: $\rho : \mathbb{R} \to (-1, 1) \quad \rho(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$



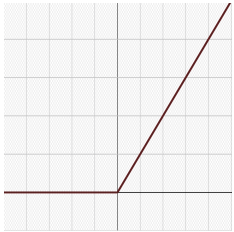
- Soboleva modified hyperbolic tangent:
  $\rho : \mathbb{R} \to (-1, 1) \quad \rho(x) = \frac{e^{ax} - e^{-bx}}{e^{cx} + e^{-dx}}$
  $a, b, c, d \in \mathbb{R}$



Saturating function usually suffer from **vanishing gradient** problem (more on that later).

Modern Activation Operators

- Rectified Linear Unit (ReLU): $\rho : \mathbb{R} \to [0, \infty) \quad \rho(x) = \max(0, x)$

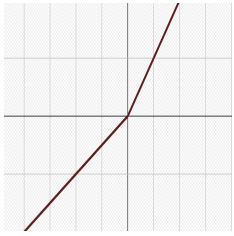


More favorable for the gradient flow, but may suffer from dying ReLU problem.

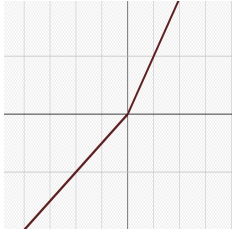
Other variants have been proposed to mitigate this problem, see next slide.

ReLU variants

- Leaky Rectified Linear Unit:
  $\rho : \mathbb{R} \to \mathbb{R}$
  $$\rho(x) = \begin{cases} 0.01x & \text{if} \quad x \leq 0 \\ x & \text{if} \quad x > 0 \end{cases}$$

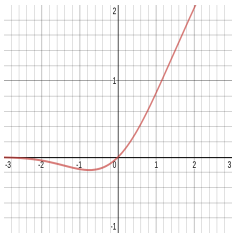


- Parametric Rectified Linear Unit:
  $\rho : \mathbb{R} \to \mathbb{R}$
  $$\rho(x) = \begin{cases} \alpha x & \text{if} \quad x \leq 0 \\ x & \text{if} \quad x > 0, \end{cases}$$
  with $\alpha \in \mathbb{R}^+$.

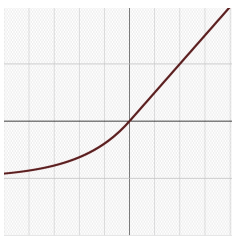


Other ReLU variants

- Gaussian Error Linear Unit (GELU):
  $\rho : \mathbb{R} \to [-0.17, \infty)$
  $\rho(x) = \frac{1}{2}x\left(1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right)\right) = x\Phi(x),$
  where $\Phi(x) = P(X \leq x), X \approx \mathcal{N}(0,1)$ is the cumulative distribution function of the normal distribution.

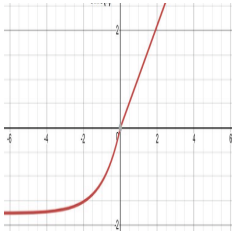


- Exponential Linear Unit (ELU):
  $\rho : \mathbb{R} \to [-\alpha, \infty)$
  $$\rho(x) = \begin{cases} \alpha(e^x - 1) & \text{if} \quad x \leq 0 \\ x & \text{if} \quad x > 0 \end{cases},$$
  where $\alpha \in \mathbb{R}^+$.

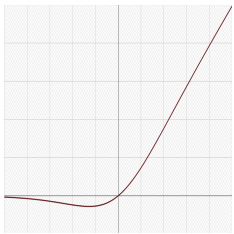


More ReLU activation functions

- Scaled Exponential Linear Unit (SELU):
  $\rho : \mathbb{R} \to [-\alpha\lambda, \infty)$
  $$\rho(x) = \lambda \begin{cases} \alpha(e^x - 1) & \text{if} \quad x \leq 0 \\ x & \text{if} \quad x > 0 \end{cases},$$
  where $\alpha = 1.67326$ and $\lambda = 1.0507$.

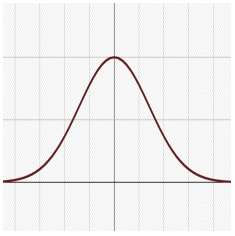


- Sigmoid Linear Unit (SiLU): $\rho : \mathbb{R} \to [-0.278, \infty)$ $\rho(x) = \frac{x}{1+e^{-x}}$,

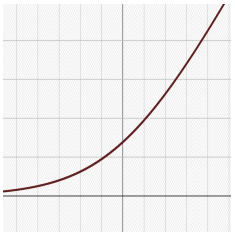


Other activation functions

- Gaussian: $\rho : \mathbb{R} \to [0, 1)$ $\rho(x) = e^{-x^2}$



- Softplus: $\rho : \mathbb{R} \to [0, \infty))$ $\rho(x) = ln(1 + e^x)$



Output activation functions

- Softmax:
  $\forall x = (x_i)_{1 \geq i \geq J} \quad \rho : \mathbb{R} \to (0,1) \quad \rho_i(x) = \frac{e^{x_i}}{\sum_{j=1}^{J} e^{x_j}}$
  Derivative: $\frac{\partial \rho_i(x)}{\partial x_j} = \rho_i(x)(\delta_{i,j} - \rho_j(x))$,
  where $\delta_{i,j}$ is the Kronecker delta.

- Maxout:
  $\forall x = (x_i)_{1 \geq i \geq J} \quad \rho : \mathbb{R} \to \mathbb{R} \quad \rho_i(x) = \max_i \quad x_i$
  Derivative: $\frac{\partial \rho_i(x)}{\partial x_j} = \begin{cases} 1 & \text{if} \quad j = \text{argmax}_i \quad (x_i) \\ 0 & \text{if} \quad j \neq \text{argmax}_i \quad (x_i) \end{cases}$

The output activation is task-dependent.

Regression

- $\rho \to$ identity
- ℓ_2 loss

Binary classification

- $\rho \to$ sigmoid
- BCE loss

Multi classification

- $\rho \to$ softmax
- CE loss

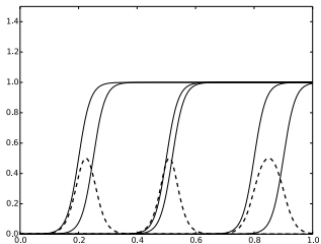
Universal Approximators

Theorem : *Any well behaved function can be arbitrarily approximated with a single layer FNN.*

Proof: A visual proof can be found in here. The original results were presented in (Cybenko, 1989).

Some intuition about this:

- Each layer uses a linear transform : $y = w^\top x$
- We can consider the sigmoid transfer function $\sigma = \rho(y) = \frac{1}{1+e^{-y}}$
- Combine multiple $\sigma-$layer activations to build gaussian kernels.



- subtract and weight consecutive $\sigma-$ layers to obtain RBF kernels, which hold the universal approximation theorem.

Why do we need deep networks?

- It is true that FFNs are universal approximators, but the hidden layer can be arbitrarily large.
- Having a large number of layers $\rightarrow$ explainable networks & helps us to build high level features. Example for an image:
  - first layer: compute contours, texure, rough features ...
  - second layer: learn corners, curves, crosses
  - next layers: build up more complex features.
- Using a shallow network $\rightarrow$ we are not able to compose low features into higher ones, we need to learn all the possibilities at once.

Error backpropagation

- Training the network is an iterative algorithm, performed by gradient descent (or its variants)
- The algorithm was first introduced by (Werbos, 1981) and then popularized by (Rumelhart and co., 1986).

Training mechanism

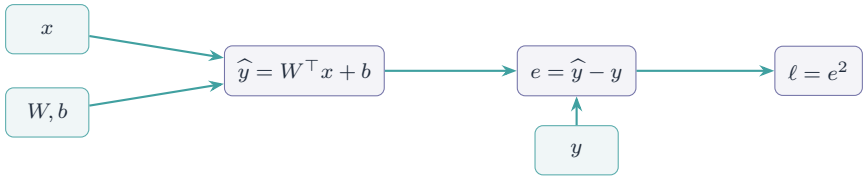
1. initialize network parameters (weights and biases - η) at $t = 0$
2. at every iteration t compute: $w_t \leftarrow w_{t-1} - \gamma \nabla_\eta \mathcal{L}$

- Usually a mini-batch version of gradient descent is used (see previous Lecture for details).
- The main question is how do we compute $\frac{\partial \mathcal{L}}{\partial w}$?

Computational graph

A **computational graph** is a directed acyclic graph whose nodes represent:

- **variables:** weights, inputs, predictions, ground truth, loss;
- **operations:** $W^\top x + b$, ReLU, Softmax, squared error, ...



For linear regression $\mathbb{R}^8 \to \mathbb{R}$, keep a minibatch of $M = 64$ pairs (x_i, y_i):

$$\widehat{y}_i = W^\top x_i + b, \qquad \boxed{\mathcal{L} = \frac{1}{M}\sum_{i=1}^{M}(\widehat{y}_i - y_i)^2 = \frac{1}{M}\sum_{i=1}^{M}(W^\top x_i + b - y_i)^2}.$$

$\widehat{y}$ always denotes a prediction; y denotes ground truth.

One layer: differentiate the loss before adding depth

For one sample, $\widehat{y} = W^\top x + b$, $e = \widehat{y} - y$ and $\ell = e^2$.

Training: differentiate with respect to W, b; keep x, y fixed

$$\frac{\partial \ell}{\partial W} = \underbrace{\frac{\partial \ell}{\partial \widehat{y}}}_{2(\widehat{y}-y)} \underbrace{\frac{\partial \widehat{y}}{\partial W}}_{x^\top} = \boxed{2(\widehat{y}-y)x^\top}, \qquad \frac{\partial \ell}{\partial b} = 2(\widehat{y}-y).$$

Average the same derivative over the minibatch:

$$\boxed{\frac{\partial \mathcal{L}}{\partial W} = \frac{2}{M}\sum_{i=1}^{M}(W^\top x_i + b - y_i)x_i^\top}.$$

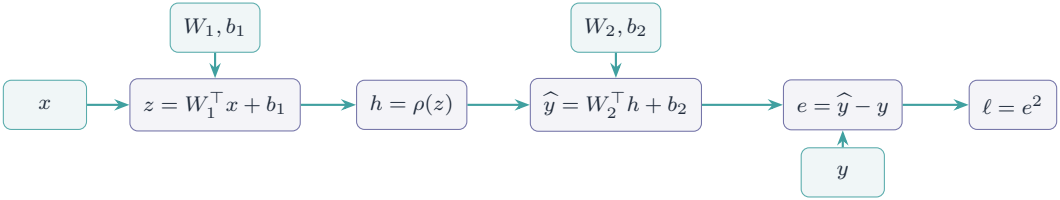
With one scalar hidden neuron: $h = \rho(W_1^\top x + b_1)$ and $\widehat{y} = W_2^\top h + b_2$ give

$$\frac{\partial \ell}{\partial W_1} = 2(\widehat{y}-y)\, W_2\, \rho'(W_1^\top x + b_1)\, x^\top.$$

An extra layer introduces extra local factors.

One scalar hidden neuron: follow the derivative path

For one sample, keep x, y fixed and differentiate with respect to W_1.



Follow the path $W_1 \to z \to h \to \widehat{y} \to e \to \ell$

$$\frac{\partial \ell}{\partial W_1} = \underbrace{\frac{\partial \ell}{\partial e}}_{2e} \underbrace{\frac{\partial e}{\partial \widehat{y}}}_{1} \underbrace{\frac{\partial \widehat{y}}{\partial h}}_{W_2} \underbrace{\frac{\partial h}{\partial z}}_{\rho'(z)} \underbrace{\frac{\partial z}{\partial W_1}}_{x^\top}$$

$$= \boxed{2(\widehat{y} - y)\, W_2\, \rho'(z)\, x^\top}.$$

Computational graph: derivative conventions

Let η contain all variables updated during training: weights and biases.

- **Problem:** compute the partial derivatives of $\mathcal{L}$ with respect to η.
- **Solution:** compute local derivatives and combine them using the **chain rule**.

We now use derivatives with respect to x to explain the chain rule; the training objective still concerns η.

Numerator layout: our convention

For $\widehat{y} = f(x) \in \mathbb{R}^{n_y}$, $x \in \mathbb{R}^{n_x}$,

$$[J_{\widehat{y},x}]_{ij} = \frac{\partial \widehat{y}_i}{\partial x_j}, \quad J_{\widehat{y},x} \in \mathbb{R}^{n_y \times n_x}.$$

Outputs index rows; inputs index columns.

Denominator layout

The same entries are transposed:

$$K_{\widehat{y},x} = J_{\widehat{y},x}^{\top} \in \mathbb{R}^{n_x \times n_y}.$$

Inputs index rows; outputs index columns.

For scalar $\mathcal{L}$ and $W \in \mathbb{R}^8$, $\partial \mathcal{L}/\partial W \in \mathbb{R}^{1\times 8}$, whereas $\nabla_W \mathcal{L} = (\partial \mathcal{L}/\partial W)^{\top} \in \mathbb{R}^8$.

Both are Jacobian conventions. We keep numerator layout throughout; a Hessian contains second derivatives.

The chain rule: the Jacobian matrix

Input sensitivity: vary x and hold W, b fixed.
In numerator layout, the derivative of a scalar with respect to a vector is a **row vector**:

$$\mathcal{L} \in \mathbb{R}, \quad x \in \mathbb{R}^{n_x}, \qquad \frac{\partial \mathcal{L}}{\partial x} \in \mathbb{R}^{1 \times n_x}.$$

For $\widehat{y} : \mathbb{R}^{n_x} \to \mathbb{R}^{n_y}$, its Jacobian is an $n_y \times n_x$ matrix:

$$\frac{\partial \widehat{y}}{\partial x}(x) = \begin{bmatrix} \frac{\partial \widehat{y}_1}{\partial x_1} & \frac{\partial \widehat{y}_1}{\partial x_2} & \cdots & \frac{\partial \widehat{y}_1}{\partial x_{n_x}} \\ \frac{\partial \widehat{y}_2}{\partial x_1} & \frac{\partial \widehat{y}_2}{\partial x_2} & \cdots & \frac{\partial \widehat{y}_2}{\partial x_{n_x}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial \widehat{y}_{n_y}}{\partial x_1} & \frac{\partial \widehat{y}_{n_y}}{\partial x_2} & \cdots & \frac{\partial \widehat{y}_{n_y}}{\partial x_{n_x}} \end{bmatrix} (x)$$

For $\widehat{y} = W^\top x + b$, use $W \in \mathbb{R}^{n_x \times n_y}$, so $J_{\widehat{y},x} = W^\top$.

Symbolic differentiation: work out the entries

$$W = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix}, \qquad \widehat{y} = W^\top x + b = \begin{bmatrix} w_{11}x_1 + w_{21}x_2 + b_1 \\ w_{12}x_1 + w_{22}x_2 + b_2 \end{bmatrix}.$$

Differentiate with respect to x, holding W, b (and the target y) fixed:

$$\frac{\partial \widehat{y}}{\partial x} = \begin{bmatrix} \frac{\partial \widehat{y}_1}{\partial x_1} & \frac{\partial \widehat{y}_1}{\partial x_2} \\ \frac{\partial \widehat{y}_2}{\partial x_1} & \frac{\partial \widehat{y}_2}{\partial x_2} \end{bmatrix} = \boxed{\begin{bmatrix} w_{11} & w_{21} \\ w_{12} & w_{22} \end{bmatrix} = W^\top}.$$

Now differentiate the loss expression symbolically

$$\ell = (w_{11}x_1 + w_{21}x_2 + b_1 - y_1)^2 + (w_{12}x_1 + w_{22}x_2 + b_2 - y_2)^2,$$

$$\frac{\partial \ell}{\partial x_1} = 2(w_{11}x_1 + w_{21}x_2 + b_1 - y_1)w_{11} + 2(w_{12}x_1 + w_{22}x_2 + b_2 - y_2)w_{12}.$$

The chain rule: vectors and matrices

For a chain $u_1 \to u_2 = \rho_1(u_1) \to \cdots \to u_k = \rho_{k-1}(u_{k-1})$,

$$\frac{\partial u_k}{\partial u_1} = \frac{\partial u_k}{\partial u_{k-1}} \frac{\partial u_{k-1}}{\partial u_{k-2}} \cdots \frac{\partial u_2}{\partial u_1}.$$

Input sensitivity: recover the preceding symbolic derivative

With $e = \widehat{y} - y$, $\ell = e^\top e$ and $\widehat{y} = W^\top x + b$,

$$\underbrace{\frac{\partial \ell}{\partial x}}_{1 \times n_x} = \underbrace{2e^\top}_{1 \times n_y} \underbrace{W^\top}_{n_y \times n_x}.$$

For the 2×2 example: $\dfrac{\partial \ell}{\partial x} = [\, 2e_1 w_{11} + 2e_2 w_{12} \quad 2e_1 w_{21} + 2e_2 w_{22} \,]$.

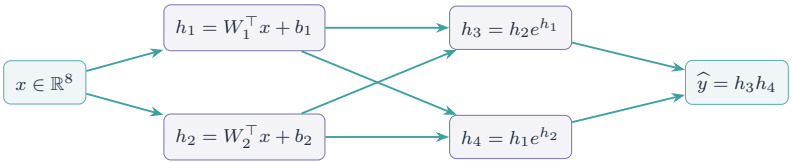
Training gradient: differentiate with respect to W; keep x, y fixed

Since $\widehat{y}_j = \sum_i W_{ij} x_i + b_j$,

$$\frac{\partial \ell}{\partial W_{ij}} = 2e_j x_i \quad \Longrightarrow \quad \boxed{\nabla_W \ell = 2xe^\top}, \qquad \nabla_W \ell = 2 \begin{bmatrix} x_1 e_1 & x_1 e_2 \\ x_2 e_1 & x_2 e_2 \end{bmatrix} \text{ in our example.}$$

Multiple paths

To obtain the final derivative, **sum the contributions of all dependency paths**.



The two affine branches have different input derivatives:

$$\frac{\partial h_1}{\partial x} = W_1^\top, \qquad \frac{\partial h_2}{\partial x} = W_2^\top.$$

$$\frac{\partial \widehat{y}}{\partial x} = h_4 \underbrace{\left(h_2 e^{h_1} W_1^\top + e^{h_1} W_2^\top\right)}_{\partial h_3/\partial x} + h_3 \underbrace{\left(e^{h_2} W_1^\top + h_1 e^{h_2} W_2^\top\right)}_{\partial h_4/\partial x}.$$

All four paths contribute. Weights and biases remain fixed.

Automatic differentiation

Automatic differentiation computes derivatives by applying local derivative rules and the **chain rule** to the operations in a program.

Method	What it does	What we obtain
Symbolic	Manipulates expressions using algebraic differentiation rules.	A derivative expression, such as the expanded formula on the symbolic-example slide.
Finite differences	Perturbs an input and evaluates the function, e.g. $\frac{f(x+\varepsilon)-f(x-\varepsilon)}{2\varepsilon}$.	An approximation affected by step size and rounding.
Automatic (AD)	Evaluates local derivatives and composes them along the executed computation graph.	Derivative values at the supplied inputs, up to numerical precision.

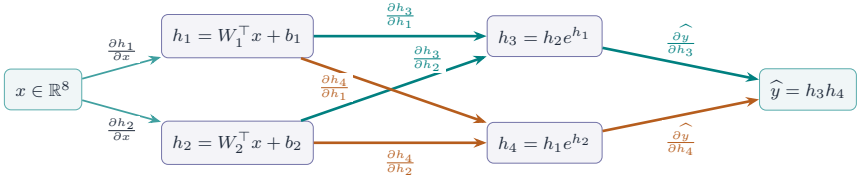
Two modes, using the same chain rule

Forward mode: propagate input sensitivities towards the output.
Reverse mode: propagate output sensitivities towards the inputs and parameters.

Reverse mode is especially useful for many parameters and one scalar loss.

For more details, see Margossian's review of automatic differentiation (2019).

Automatic differentiation – example: forward mode



Follow the forward dependency order. Arrow labels show **local partial derivatives**.

1. Differentiate the affine nodes

$$\frac{\partial h_1}{\partial x} = W_1^\top, \qquad \frac{\partial h_2}{\partial x} = W_2^\top.$$

2. Differentiate each branch

$$\frac{\partial h_3}{\partial x} = e^{h_1}\left(\frac{\partial h_2}{\partial x} + h_2 \frac{\partial h_1}{\partial x}\right),$$
$$\frac{\partial h_4}{\partial x} = e^{h_2}\left(\frac{\partial h_1}{\partial x} + h_1 \frac{\partial h_2}{\partial x}\right).$$

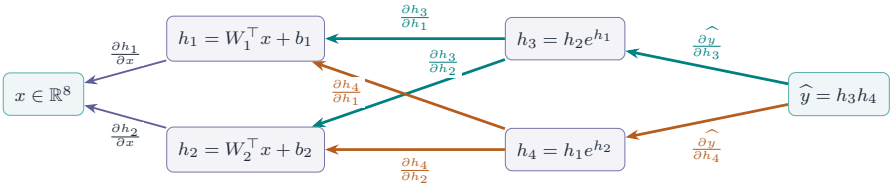
3. Apply the product rule at the output:

$$\frac{\partial \widehat{y}}{\partial x} = h_4 \frac{\partial h_3}{\partial x} + h_3 \frac{\partial h_4}{\partial x} = h_4 e^{h_1}(W_2^\top + h_2 W_1^\top) + h_3 e^{h_2}(W_1^\top + h_1 W_2^\top).$$

For one input direction s, propagate the scalars $\left(\frac{\partial h_i}{\partial x}\right) s$.

Automatic differentiation – example: reverse mode

Start with $\frac{\partial \widehat{y}}{\partial \widehat{y}} = 1$ and propagate from right to left.



1. From the output

$$\frac{\partial \widehat{y}}{\partial h_3} = h_4, \qquad \frac{\partial \widehat{y}}{\partial h_4} = h_3.$$

2. Add the contributions from both branches

$$\frac{\partial \widehat{y}}{\partial h_1} = \frac{\partial \widehat{y}}{\partial h_3} h_2 e^{h_1} + \frac{\partial \widehat{y}}{\partial h_4} e^{h_2},$$

$$\frac{\partial \widehat{y}}{\partial h_2} = \frac{\partial \widehat{y}}{\partial h_3} e^{h_1} + \frac{\partial \widehat{y}}{\partial h_4} h_1 e^{h_2}.$$

3. Reach the input:

$$\boxed{\frac{\partial \widehat{y}}{\partial x} = \frac{\partial \widehat{y}}{\partial h_1} W_1^\top + \frac{\partial \widehat{y}}{\partial h_2} W_2^\top}.$$

For $\ell = (\widehat{y} - y)^2$, multiply by $2(\widehat{y} - y)$ to obtain derivatives of the loss.

Error backpropagation – two-layer modelling example

For one sample (x, y), keep the affine convention at **both** layers:

$$z_1 = W_1^\top x + b_1, \quad h = \rho(z_1), \quad \widehat{y} = W_2^\top h + b_2, \quad \ell = \phi(y, \widehat{y}).$$

$W_1 \in \mathbb{R}^{n_x \times n_1}$, $W_2 \in \mathbb{R}^{n_1 \times n_2}$; ρ acts elementwise. For squared error, $\phi(y, \widehat{y}) = \|\widehat{y} - y\|^2$.

Propagate column sensitivities backwards

$$\delta_2 = \nabla_{\widehat{y}} \ell = 2(\widehat{y} - y),$$
$$\delta_1 = \nabla_{z_1} \ell = (W_2 \delta_2) \odot \rho'(z_1).$$

Form the parameter gradients

$$\nabla_{W_2} \ell = h \delta_2^\top, \qquad \nabla_{b_2} \ell = \delta_2,$$
$$\nabla_{W_1} \ell = x \delta_1^\top, \qquad \nabla_{b_1} \ell = \delta_1.$$

Extend to more layers

At each hidden layer, $z_l = W_l^\top a_{l-1} + b_l$ and $a_l = \rho_l(z_l)$ give

$$\delta_l = (W_{l+1} \delta_{l+1}) \odot \rho_l'(z_l), \qquad \nabla_{W_l} \ell = a_{l-1} \delta_l^\top.$$

Practical example using PyTorch

Most deep-learning frameworks calculate derivatives automatically using the recorded computation graph.

Our one-layer model, written exactly as before

```
# x: (nx,), W: (nx, ny)
# b, y: (ny,)
y_hat = W.T @ x + b
loss = ((y_hat - y) ** 2).sum()
loss.backward()
# W.grad, b.grad: parameter gradients
```

Enable gradient tracking for W, b; enable it for x only when input sensitivities are needed. Clear accumulated gradients before the next training step.

Recording and execution

- **PyTorch eager:** autograd records tracked operations as the forward pass runs.
- **TensorFlow 2 eager:** `GradientTape` records relevant operations in its context.
- `torch.compile` and `tf.function` also support compiled graph execution.

Forward: compute predictions and loss. Backward: compute derivatives. The optimizer then updates the parameters.

Computational graphs on a GPU

The mathematical dependencies and the chain rule stay the same

What the framework does

- Records or compiles the operations and their dependencies.
- Schedules CPU/GPU kernels and transfers.
- Saves forward values needed during the backward pass.

What the GPU accelerates

- Parallel arithmetic within matrix products, activations and reductions.
- Many samples in a minibatch at once.
- Both forward operations and their backward computations.

With samples as columns, $X \in \mathbb{R}^{n_x \times M}$ and $W \in \mathbb{R}^{n_x \times n_y}$:

$$\widehat{Y} = W^\top X + b\mathbf{1}^\top, \quad E = \widehat{Y} - Y, \quad \mathcal{L} = \frac{1}{M}\|E\|_F^2, \qquad \boxed{\nabla_W \mathcal{L} = \frac{2}{M} X E^\top}.$$

A layer still needs its inputs: not all layers run simultaneously. GPU launches are usually asynchronous with the CPU; operations in one CUDA stream remain ordered. Transfers and launch overhead can outweigh the benefit for tiny examples.

Which gradients do we compute, and why?

Standard supervised training: change the parameters

Keep the training data x, y fixed. Compute $\nabla_W \mathcal{L}$ and $\nabla_b \mathcal{L}$ to reduce the minibatch loss:

$$W \leftarrow W - \alpha \nabla_W \mathcal{L}, \qquad b \leftarrow b - \alpha \nabla_b \mathcal{L}.$$

Here $\alpha > 0$ is the learning rate.
Backpropagation passes through hidden activations to obtain gradients of earlier weights.

Input sensitivity: study changes to the data

Keep W, b and the target y fixed. $\nabla_x \ell$ measures local loss sensitivity; $J_{\widehat{y},x}$ measures prediction sensitivity.

For **robustness**, input gradients guide a search for small allowed perturbations that increase the loss:

$$\max_{\delta \in \Delta} \ \phi\big(y, f_{W,b}(x+\delta)\big).$$

Here Δ specifies the allowed input changes.

For the same model $\widehat{y} = W^\top x + b$, $e = \widehat{y} - y$ and $\ell = e^\top e$:

$$\underbrace{\nabla_W \ell = 2xe^\top, \qquad \nabla_b \ell = 2e,}_{\text{parameter gradients for training}} \qquad \underbrace{\nabla_x \ell = 2We}_{\text{input gradient for sensitivity}} \quad .$$

The chain rule is the same; the differentiation variable determines the question we answer.
Standard training does not require the raw-input gradient. Adversarial training uses both: input gradients to construct examples, parameter gradients to train on them.

Exercise: one training step by hand

Consider a network with two inputs, two ReLU hidden neurons and one scalar output:

$$z = W_1^\top x + b_1, \qquad h = \mathrm{ReLU}(z), \qquad \widehat{y} = W_2^\top h + b_2.$$

$$x = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad y = 0, \qquad W_1 = \begin{bmatrix} 1 & -1 \\ 1 & 1 \end{bmatrix}, \quad b_1 = \begin{bmatrix} -1 \\ -1 \end{bmatrix}, \quad W_2 = \begin{bmatrix} 1 \\ 2 \end{bmatrix}, \quad b_2 = 0.$$

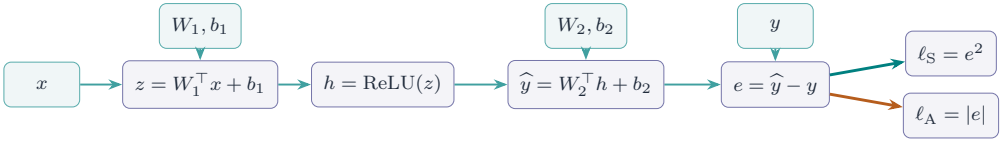
Compare two losses on this same sample

$$\ell_{\mathrm{S}} = (\widehat{y} - y)^2 \quad \text{(squared error)}, \qquad \ell_{\mathrm{A}} = |\widehat{y} - y| \quad \text{(absolute error)}.$$

1. Draw the computational graph, including weights, biases, the target and both losses.
2. Compute $z, h, \widehat{y}$ and the two loss values.
3. For **each loss**, compute $\nabla_x \ell$, $\nabla_{W_1} \ell$, $\nabla_{b_1} \ell$, $\nabla_{W_2} \ell$ and $\nabla_{b_2} \ell$.
4. With learning rate $\alpha = 0.1$, update **all parameters simultaneously**, then recompute the prediction and the corresponding loss.

Restart from the given parameters for each loss. Keep x, y fixed during both training steps.
Which hidden neuron receives zero gradient, and why? Why do we compute $\nabla_x \ell$ without updating x?

Correction 1: graph, forward pass and derivatives



$$z = (1, -1)^\top, \quad h = (1, 0)^\top, \quad \widehat{y} = 1, \quad e = 1, \quad \ell_{\mathrm{S}} = \ell_{\mathrm{A}} = 1.$$

Use the output sensitivity $g = \partial \ell / \partial \widehat{y}$:

$$g_{\mathrm{S}} = 2e = 2, \qquad g_{\mathrm{A}} = \mathrm{sign}(e) = 1, \qquad \delta = \nabla_z \ell = (W_2 g) \odot (1, 0)^\top = (g, 0)^\top.$$

The same backward pass, with $g = 2$ for squared error or $g = 1$ for absolute error

$$\nabla_{W_1} \ell = x\delta^\top = \begin{bmatrix} g & 0 \\ g & 0 \end{bmatrix}, \quad \nabla_{b_1} \ell = \delta = \begin{bmatrix} g \\ 0 \end{bmatrix}, \quad \nabla_{W_2} \ell = hg = \begin{bmatrix} g \\ 0 \end{bmatrix}, \quad \nabla_{b_2} \ell = g.$$

$$\nabla_x \ell = W_1 \delta = (g, g)^\top \quad \Rightarrow \quad \nabla_x \ell_{\mathrm{S}} = (2, 2)^\top, \quad \nabla_x \ell_{\mathrm{A}} = (1, 1)^\top.$$

The second hidden neuron is inactive: its incoming parameters and its outgoing weight have zero gradient here.

Correction 2: simultaneous updates and new losses

Apply $\theta^+ = \theta - 0.1\nabla_\theta \ell$ to **each parameter**, using gradients at the original values.

Squared error: $g = 2$

$$W_1^+ = \begin{bmatrix} 0.8 & -1 \\ 0.8 & 1 \end{bmatrix}, \qquad W_2^+ = \begin{bmatrix} 0.8 \\ 2 \end{bmatrix},$$

$$b_1^+ = \begin{bmatrix} -1.2 \\ -1 \end{bmatrix}, \qquad b_2^+ = -0.2.$$

Recompute the forward pass with these values:

$$z^+ = \begin{bmatrix} 0.4 \\ -1 \end{bmatrix}, \qquad h^+ = \begin{bmatrix} 0.4 \\ 0 \end{bmatrix},$$

$$\widehat{y}^+ = 0.8 \cdot 0.4 - 0.2 = \boxed{0.12},$$

$$\ell_{\mathrm{S}}^+ = (0.12 - 0)^2 = \boxed{0.0144}.$$

Absolute error: $g = 1$

$$W_1^+ = \begin{bmatrix} 0.9 & -1 \\ 0.9 & 1 \end{bmatrix}, \qquad W_2^+ = \begin{bmatrix} 0.9 \\ 2 \end{bmatrix},$$

$$b_1^+ = \begin{bmatrix} -1.1 \\ -1 \end{bmatrix}, \qquad b_2^+ = -0.1.$$

Recompute with the updated parameters:

$$z^+ = \begin{bmatrix} 0.7 \\ -1 \end{bmatrix}, \qquad h^+ = \begin{bmatrix} 0.7 \\ 0 \end{bmatrix},$$

$$\widehat{y}^+ = 0.9 \cdot 0.7 - 0.1 = \boxed{0.53},$$

$$\ell_{\mathrm{A}}^+ = |0.53 - 0| = \boxed{0.53}.$$

Both losses decrease from 1 in their separate experiment. The input x and target y stay unchanged.
The input gradient measures sensitivity; this training step updates the model parameters.

Why do we use it in deep learning

- Neural networks $\rightarrow$ **non-convex optimization**
- Using Gradient Descent $\rightarrow$ guarantee to get a **local minima**
- *Can we do better?*
  Maybe yes, but for large/deep nets it has been proven (empirically though) that most local minima are close (in performance) with the global one.
- Saddle points are critical!!

Mini-batch gradient descent

Algorithm 1: Minibatch Gradient Descent

Input: Learning rate α, minibatch size M, training dataset pairs (x_i, y_i)
Output: Updated model parameters θ

for *every minibatch* $B \subseteq D$ *of size* M **do**
    initialize (randomly) parameters θ_0;
    for *iteration* $t = 0 \dots$ **do**
        $\theta_{t+1} = \theta_t - \alpha \nabla_\theta \mathcal{L}(\theta_t)$;
        $\mathcal{L}(\theta) = \frac{1}{M} \sum_i \phi(\theta, x_i, y_i)$

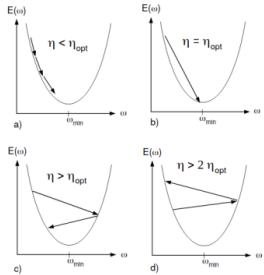
Taylor series expansion : $\mathcal{L}(\theta_{t+1}) \approx \mathcal{L}(\theta_t) + (\theta_{t+1} - \theta_t)^\top \nabla_\theta \mathcal{L}(\theta_t)$

Is the size of M important?

- If $(M = 1)$ Stochastic Gradient Descent : noisy estimate, not GPU friendly
- If $(M = N > 1)$ Batch Gradient Descent : generalization gap and local minima problem.

How about the learning rate?

The model may converge very slowly or even diverge if α is not appropriately chosen! So, what is the optimal learning rate?



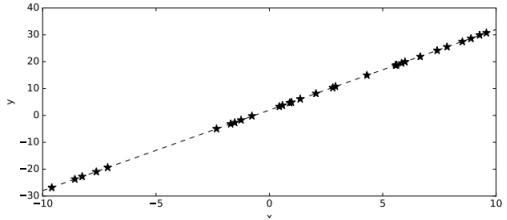
Efficient Backprop, LeCun et al., 1998

- Y.Bengio : *"The optimal learning rate is usually close to the largest learning rate that does not cause divergence of the training criterion."*
- Karpathy : *"0.0003 is the best learning rate for Adam, hands down."*

Let us take a simple example

Let us consider a **regression problem** having the following setup

- $N = 30$ samples generated with : $y = 3x + 2 + \mathcal{U}(-0.1, 0.1)$
- Model: $\rho(x) = \theta^\top \begin{bmatrix} 1 \\ x \end{bmatrix}$,
- ℓ_2 loss :$\mathcal{L}(y_i, \rho(x_i)) = (y_i - \rho(x_i))^2$

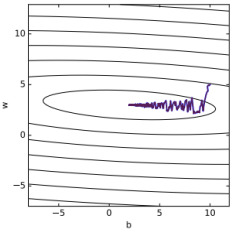


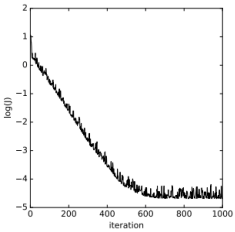
SGD minimization

Consider:

- learning rate : $\alpha = 0.005$,
- Initial parameters : $\theta_0 = \begin{bmatrix} 10 \\ 5, \end{bmatrix}$
- $t = 1000$ iterations.

Obtained solution : $\theta_{1000} = \begin{bmatrix} 1.9882 \\ 2.9975 \end{bmatrix}$





SGD with Momentum

Idea : Dampen the oscillations in the optimization path using a low pass filter on ∇_θ.

Algorithm 2: Minibatch Stochastic Gradient Descent with Momentum

Input: Learning rate α, momentum parameter β, minibatch size M, training dataset pairs (x_i, y_i)
Output: Updated model parameters θ

initialize parameters θ_0; initialize momentum vector $v_0 = 0$;
for *every minibatch* $B \subseteq D$ *of size* M **do**
  for *every iteration* $t = 0 \dots$ **do**
    $v_{t+1} = \beta v_t - \alpha \nabla_\theta J(\theta_t)$;
    $\theta_{t+1} = \theta_t + v_{t+1}$;

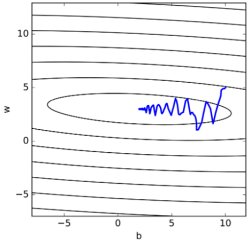
Here-above, $\beta \in \mathbb{R}$ is a hyper-parameter, usually ≈ 0.9.

- The purpose is to accelerate the learning rate in constant directions and low curvature.
- Check this link for more details about this.

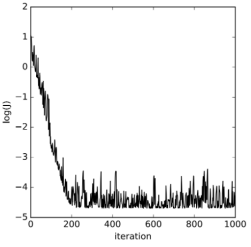
Back to our example

Parameters: $\alpha = 0.005,\ \beta = 0.6,\ \theta_0 = \begin{bmatrix} 10 \\ 5 \end{bmatrix}$

Solution : $\theta_{1000} = \begin{bmatrix} 1.9837 \\ 2.9933 \end{bmatrix}$



The optimization path



The value of the function

Gradient descent with Nesterov momentum

Idea : Based on Nesterov Accelerated Gradient. Look ahead to correct the update.

Algorithm 3: Minibatch Stochastic Gradient Descent with Nesterov Momentum

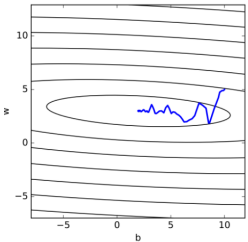
Input: Learning rate α, momentum parameter β, minibatch size M, training dataset pairs (x_i, y_i)
Output: Updated model parameters θ

initialize parameters θ_0; initialize momentum vector $v_0 = 0$;
for *every minibatch* $B \subseteq D$ *of size* M **do**
    for *every iteration* $t = 0 \dots$ **do**
        $v_{t+1} = \beta v_t - \alpha \nabla_\theta J(\theta_t + \beta v_t)$; (Nesterov lookahead update)
        $\theta_{t+1} = \theta_t + v_{t+1}$;

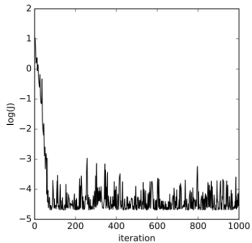
Back to our example

Parameters: $\alpha = 0.005,\ \beta = 0.8,\ \theta_0 = \begin{bmatrix} 10 \\ 5 \end{bmatrix}$

Solution : $\theta_{1000} = \begin{bmatrix} 1.9738 \\ 2.9914 \end{bmatrix}$

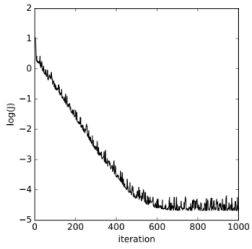


The optimization path

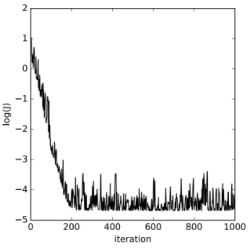


The value of the function

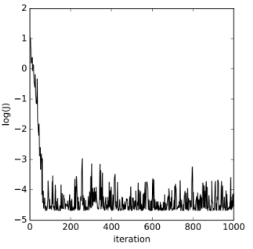
Comparison



SGD



Momentum



Nesterov

Adaptive Learning rate

Idea : Modify the learning rate during the training phase

Decreasing the learning rate – Robbins Monro conditions

- Linear decrease from α_0 to α_f
- Halve the learning rate when the validation error stops improving
- Halve the learning rate on a fixed schedule

More recent approaches are not advocating for decreasing the learning rate

Increasing learning rate

- The 1-cycle policy
- Stochastic Gradient Descent with Warm Restart

Adaptive first order : ADAGRAD

Adagrad - Adaptive gradient proposed in 2011

- Accumulate the square of the gradient

$$r_{t+1} = r_t + \nabla_\theta J(\theta_t) \odot \nabla_t J(\theta_t)$$

- Scale individually the learning rates

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\delta + \sqrt{r_{t+1}}} \odot \nabla_\theta J(\theta_t),$$

where $\delta \approx 1e^{-8}$

- Small gradients $\rightarrow$ bigger learning rate for moving fast along flat directions
- High gradients $\rightarrow$ smaller learning rate to calm down on high curvature

RMSProp

Idea : using an exponential moving average when accumulating the gradient.

- Accumulate the square of the gradient:

$$r_{t+1} = \rho r_t + (1 - \rho)\nabla_\theta J(\theta_t) \odot \nabla_\theta J(\theta_t)$$

- Scale each learning rate accordingly:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\delta + \sqrt{r_{t+1}}} \odot \nabla_\theta J(\theta_t),$$

where ρ is a training parameter, $\rho \approx 0.9$

ADAM algorithm

Idea : Use Adaptive Moments of past gradients

- Store running averages of past gradients:

$$m_{t+1} = \beta_1 m_t + (1 - \beta_1)\nabla_\theta J(\theta_t)$$

$$v_{t+1} = \beta_2 v_t + (1 - \beta_2)\nabla_\theta J(\theta_t) \odot \nabla_\theta J(\theta_t),$$

$m(t)$ and $v(t)$ are the first and second (uncentred) moments of $\nabla_\theta J$. Correct the bias $\hat{m}_t$ and $\hat{v}_t$:

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\delta + \sqrt{\hat{v}_{t+1}}}\hat{m}_{t+1}$$

Some references

- Overview of gradient descent
- Recent gradient descent algorithms

First order methods summary

Goodfellow, Bengio, & Courville – " There is currently no consensus [...] no single best algorithm has emerged [...] "

- The most popular and actively in use include SGD, SGD with momentum, RMSprop, RMSprop with momentum, Adadelta and Adam
- It is task-specific mostly.

Second order methods

Idea : use the second derivative, and Taylor series expansion

$$J(\theta_t) \approx J(\theta_0) + (\theta - \theta_0)^\top \nabla_\theta J(\theta_0) + \frac{1}{2}(\theta - \theta_0)^\top \nabla^2_\theta J(\theta_0)(\theta - \theta_0),$$

where $H = \nabla^2 J$ - Hessian matrix

Methods

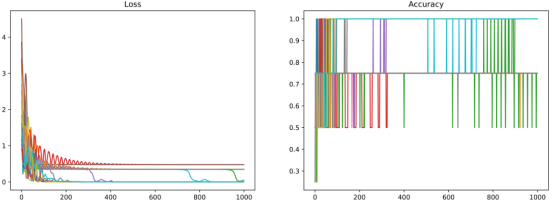
- Conjugate gradient : using line search along $\nabla_\theta J(\theta_k)$
- Newton : just to find critical points
- Quasi Newton : BFGS (approximating H^{-1}), L-BFGS, and saddle-free versions

Does the starting point matter?

Problem : XOR classification

Model architecture :

- Layers: 3 Dense (2-4-1); Activation: Sigmoid
- Init: $\mathcal{U}(-10, 10)$, bias $= 0$
- Loss: BCE
- SGD ($\alpha = 0.1$, momentum $= 0.99$)
- $n = 1000$ epochs



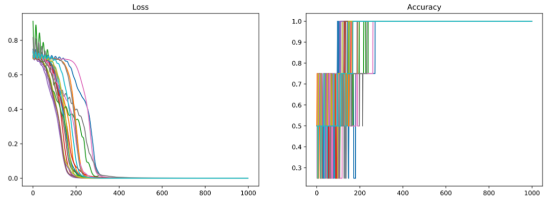
It fails 6/20 times.

Does the starting point matter?

Problem : XOR classification

Model architecture :

- Layers: 3 Dense (2-4-1); Activation: Sigmoid
- Init: $\mathcal{N}(0, \frac{1}{\sqrt{fan_{in}}})$, bias $= 0$
- Loss: BCE
- SGD ($\alpha = 0.1$, momentum $= 0.99$)
- $n = 1000$ epochs



It fails 0/20 times.

Standardization

- The starting point of a gradient descent has a dramatic impact.
- All gradient-based algorithms converge faser is the data are normalized and decorrelated.
- Several strategies are currently used, depending on the task.

Let x be you input data.

Min-Max scaling

$$x' = \frac{x - \min\ (\mathrm{x})}{\max(x) - \min(x)}$$

Z-score scaling

$$x' = \frac{x - \mu_x}{\sigma_x + \epsilon},$$

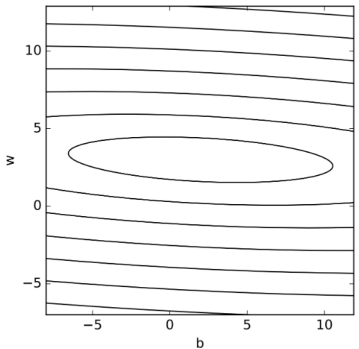
where μ_x, σ_xandϵ are the mean, the standard deviation and a small constant, respectively.

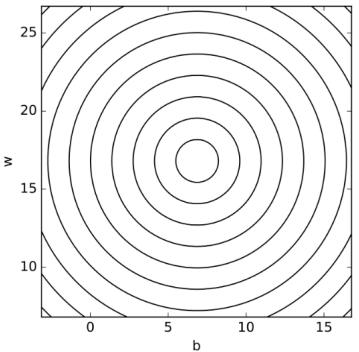
ZCA whitening

$$X' = WX, W = \frac{1}{\sqrt{n-1}}(X'X^\top)^{(-1/2)}$$

The effect of standardization

Problem: $y = 3x + 2 + \mathcal{U}(-0.1, 0.1)$





How to chose a good initialization

- An effective initialization strategy should disrupt symmetry within the neural network; constant initialization methods tend to lead units to learn identical patterns.
- A favorable initialization should commence optimization from a region of limited capacity, e.g. linear neural networks.
- An ideal initialization scheme should maintain the distribution of both activations and gradients, guarding against issues such as exploding or vanishing gradients.

Exploding and vanishing gradients

- **The Fundamental Deep Learning Problem** – the gradient can either vanish or explode, especially in deep networks.
- The deeper the network ($m \rightarrow$ large), the more chances that through backprop the gradients either explode or vanish.
- Problems with activations such as *sigmoid*, *ReLU*.

Preventing vanishing/ exploding gradients

- ensure a good flow of gradient – use appropriate activation functions (e.g. ReLu, PreLu, etc..) or some architectural elements (e.g. skip-connections, constant error loops (LSTMs))
- use *clipping* methods $\rightarrow$ only for exploding gradients problem

Initialization of Y. LeCun

First introduced in Gradient-Based Learning Aplied to Document Recognition

Goal : Initialize parameters such that the activation operator ρ stays in its linear part by using more layers:

- use symmetric transfer operations : R_i is symmetric, which means there exist $(c_i, d_i) \in (\mathbb{R}^{N_i})^2$ such that
$$(\forall x \in \mathbb{R}^{N_i}) \quad R_i(x) - d_i = -R_i(-x + c_i).$$
e.g. $R_i = 1.7159\tanh(\frac{2}{3}x)$
- initialize the biases with 0 and the weights drawn randomly from $\mathcal{N}(\mu = 0, \sigma^2 = \frac{1}{fan_{N_i}})$
If $x \in \mathbb{R}^n$ is $\mathcal{N}(0, \Sigma = I_d)$ and $w \in \mathbb{R}^n$ is $\mathcal{N}(0, \Sigma = \frac{1}{n} I_d)$. Assume x_i, w_i pair wise independent, then:

$$E[w^\top x + b] = E[w^\top x] = \sum_i E[w_i x_i] \sum_i E[w_i] E[x_i] = 0$$

$$\begin{aligned} \text{var}[w^\top x + b] &= \text{var}[w^\top x] \\ &= \sum_i \sigma_{w_i}^2 \sigma_{x_i}^2 + \sigma_{w_i}^2 \mu {x_i}^2 + \mu {w_i}^2 \sigma_{x_i}^2 \\ &= \sum_i \sigma_{w_i}^2 \sigma_{x_i}^2 = \frac{1}{n} \sum_i \sigma_{x_i}^2 = 1 \end{aligned}$$

Glorot Initialization

Goal : preserve the same distribution along the forward and backward pass, which prevents saturating transfer functions.

$$T = R_m(W_m \dots R_1(W_1(W_0 x + b_0) + b_1) \cdots + b_m)$$

Glorot (Xavier) initialization

- Input data should be centered, normalized and decorrelated
- Symmetric activation functions should be used, with $R_i'(0) = 1$

Assuming the linear regime $R_i'() = 1$ for the network:

- Forward prop. var. constraint : $\forall i, fan_{in_i} \sigma^2_{W_i} = 1$
- Backward prop. var. constraint : $\forall i, fan_{out_i} \sigma^2_{W_i} = 1$

Compromise: $\forall i, \frac{1}{\sigma^2_{W_i}} = \frac{fan_{in} + fan_{out}}{2}$

- Glorot Uniform: $\mathcal{U}\left(-\frac{\sqrt{6}}{\sqrt{fan_{in}+fan_{out}}}, \frac{\sqrt{6}}{\sqrt{fan_{in}+fan_{out}}}\right)$ and $b = 0$
- Glorot Normal: $\mathcal{N}\left(0, \frac{\sqrt{2}}{\sqrt{fan_{in}+fan_{out}}}\right)$ and $b = 0$

He Initialization

Goal : preserve the same distribution along the forward and backward pass for rectifiers.

$$T = R_m(W_m \dots R_1(W_1(W_0 x + b_0) + b_1) \cdots + b_m)$$

- input dimensions should be centered, normalized, uncorrelated
- use ReLU operators: $\forall i \in \{1, \dots, m\} \quad R_i(x) = \max(0, x)$
- weights are initialized with symmetric distribution, zero-mean, independently
- biases are set to 0
- the components of x are assumed i.i.d.

Forward prop. var. constraint : $\forall i, \frac{1}{2} fan_{in_i} \sigma^2_{W_i} = 1$

Backward prop. var. constraint : $\forall i, \frac{1}{2} fan_{out_i} \sigma^2_{W_i} = 1$

Compromise: use one of them, e.g. $\sigma^2_{W_i} = \frac{2}{fan_{out}}$

- He Uniform: $\mathcal{U}\left(-\frac{\sqrt{6}}{\sqrt{fan_{in}}}, \frac{\sqrt{6}}{\sqrt{fan_{in}}}\right)$ and $b = 0$
- He Normal: $\mathcal{N}\left(0, \frac{\sqrt{2}}{\sqrt{fan_{in}}}\right)$ and $b = 0$